Machine Learning Workflow Analysis

Email Classification using Scikit-Learn Pipelines

1. Overview of the Pipeline

The provided code snippet demonstrates a complete supervised learning workflow using a Scikit-Learn `Pipeline`. This architecture streamlines the process by bundling the preprocessing and modeling steps into a single object.

Pipeline Components:

- **Vectorizer:** `CountVectorizer` - Converts text data into numerical vectors.
- **Classifier:** `MultinomialNB` - A Multinomial Naive Bayes model, ideal for discrete features like word counts.

2. Step-by-Step Technical Workflow

1 Model Fitting (Training)

```
clf.fit(X_train, y_train)
```

During this phase, the `CountVectorizer` builds a vocabulary from the training text and transforms the documents into a document-term matrix. Then, the `MultinomialNB` algorithm calculates the prior probabilities of classes and the conditional probabilities of words given a class using the formula:

$$P(label \mid features) = (P(features \mid label) * P(label)) / P(features)$$

2 Performance Evaluation

```
clf.score(X_test, y_test) -> 0.98277
```

The model achieved an accuracy of approximately **98.28%**. This means the pipeline correctly classified nearly all emails in the unseen test set. This high score suggests the features (word frequencies) are highly discriminative for the labels provided.

3 Inference (Prediction)

```
clf.predict(emails) -> array([0, 1])
```

Finally, the model is used to predict new, raw data. In this specific execution, the input `emails` resulted in two classifications: **0** and **1** (likely representing 'Ham' and 'Spam' respectively).

3. Internal Mechanism

Feature	Description
Tokenization	Splitting sentences into individual words (tokens).
Stop Words	The output indicates `stop_words=None`, meaning common words like 'the' or 'and' were kept.
Alpha (Laplace)	`alpha=1.0` is used for smoothing, ensuring no probability is ever zero if a word appears in the test set but not in training.

Conclusion: The workflow represents a robust "Bag of Words" approach to NLP. By using a Pipeline, the user ensures that the same transformations applied to the training data are automatically and consistently applied to any new prediction data, preventing "data leakage."